Vardene: A Python Port of the LU MII Latvian Morphological Analyser:
Performance, Accuracy, and Engineering Trade-offs

Rihards Aleksandrs Freibergs*

May 8, 2026

Abstract

This paper presents *vardene*, a Python port of the Latvian morphological analysis library maintained by the Institute of Mathematics and Computer Science at the University of Latvia (LU MII). The upstream Java implementation (9,922 LOC across the engine and its HTTP service layer, GPL-3.0) is the backend of `api.tezaurs.lv`, the most widely used Latvian morphological web service. The port reproduces every algorithmic component of the upstream engine (*Mijas* stem alternations, *Trie* tokenizer, *Splitting*, *Lexicon*, *Paradigm*, *Analyzer*, *Inflector*, *MarkupConverter*), the full HTTP service layer (16 routes covering analysis, disambiguation, inflection, multi-word phrase declension, personal-name inflection, and valency-tag annotation), and adds a Python-native hierarchical disambiguator — a part-of-speech CRF, a 4-character subtag CRF, a per-POS log-linear classifier, a tag-bigram Viterbi rescoring pass, a high-confidence per-form corpus override layer, and a Latvian-specific syntactic post-pass for preposition-noun agreement — that closes the gap to the production statistical tagger. On a held-out 20 % split of the gold-standard `train.txt` corpus, evaluated over five random seeds with $n \approx 3{,}500$ tokens per seed, the system achieves a tag accuracy of $\mathbf{92.51 \pm 0.29}$ %, a lemma accuracy of $\mathbf{96.73 \pm 0.29}$ %, and a POS accuracy of $\mathbf{98.75 \pm 0.16}$ %. These figures match the published Java tag accuracy of 92.8 % within seed variance — two of five seeds exceed it — and surpass the Java POS accuracy of 98.2 % by 0.55 points. The Python package is 39 % smaller in source size, has a 2.6× faster cold start, and processes ∼1,210 tokens per second on a single CPU core after a 15× sparsification of the classifier weights that also reduces resident memory from 1.7 GB to 475 MB. The paper analyses where the Python port wins (POS accuracy, source compactness, startup latency, shipped reference-data size), where it loses (peak memory), and quantifies the contribution of each step in the disambiguator pipeline.

Keywords: Latvian, morphological analysis, lemmatisation, conditional random fields, language port, under-resourced languages, tokenisation, finite-state morphology.

1 Introduction

The Latvian morphological library at `https://github.com/PeterisP/morphology` [5] is the most thoroughly engineered open-source morphological analyser for Latvian. Released under GPL-3.0 and authored primarily by Pēteris Paikens at LU MII, the Java implementation provides the analytical backbone for `api.tezaurs.lv`, the public morphology API that Latvian natural-language-processing researchers and downstream tooling rely on. This paper describes a complete Python port of that engine and of the HTTP service layer that exposes it.

The motivation is twofold. First, the port removes deployment friction for Python-native NLP pipelines that need Latvian morphology: there is no JVM to boot, no Maven dependency tree to resolve, and no inter-process boundary between the morphology component and the rest of the

*Independent. Code: `https://github.com/freibergs/vardene`.

pipeline. Second, the port serves as a controlled laboratory for studying the engineering trade-offs of moving a mature corpus-linguistic codebase between languages — which patterns translate cleanly, which idioms become bottlenecks, and what the resulting accuracy, performance, and footprint envelope looks like.

Scope. Two upstream Java repositories were ported. `PeterisP/morphology` (the morphology engine itself, approximately 9.3 thousand lines of Java code) ships the algorithmic core: `Analyzer.analyze()` returns a candidate set of readings ranked by an additive scoring rule (`Word.getBestWordform()`). `LUMII-AILab/Webservices` [6] (approximately 3 thousand lines of Java code, also GPL-3.0) is the HTTP service layer that exposes the engine at `api.tezaurs.lv`; it wraps the engine in sixteen routes covering analysis, disambiguation, inflection, multi-word phrase declension, personal-name inflection, paradigm-suitability scoring, and valency-tag annotation.

The statistical tagger that backs the `/morphotagger` endpoint of `api.tezaurs.lv` — and that contributes the published 92.8 % tag accuracy — is a third, separate project, `LVTagger` [4], which consumes the morphology library's output and re-ranks it through a sentence-level CMM/CRF tagger. This paper ports the morphology engine and the HTTP service layer in full, and additionally trains a Python-native sentence-level disambiguator from the same gold corpus on which LVTagger was trained, since LVTagger itself is not part of either Java repository.

2 Related work

Three other open-source toolchains expose Latvian morphology to Python-native NLP pipelines, each making a different scope choice. **UDPipe** [1] ships a single end-to-end neural pipeline (tokeniser, tagger, lemmatiser, parser) trained on the Latvian Universal Dependencies treebank. **Stanza** [2] offers a similar pipeline with the same training data. Both treat morphology as a black box: they emit UD-style features per token but do not expose paradigm tables, inflection, or the candidate-set semantics that downstream tools relying on `api.tezaurs.lv` consume. **Apertium-LV** [3] ships a finite-state Latvian morphology in the Apertium framework but lacks the disambiguator stack and the lexicon breadth (411k lexemes) that LU MII curates.

The closest comparison point is **LVTagger** [4], a sentence-level CMM/CRF tagger trained on the same `train.txt` gold corpus used here. LVTagger consumes the output of the LU MII morphology library as its candidate set, the same role played by vardene's Python disambiguator. The port does not replicate LVTagger's CMM architecture; instead it uses a 4-character subtag CRF, a per-POS log-linear classifier, and a tag-bigram Viterbi rescoring pass (Section 3.3). Direct head-to-head comparison against UDPipe and Stanza is left to future work because the training data, tagsets, and evaluation protocols all differ.

3 Architecture

3.1 Components ported one-to-one

The compactness gain comes from three sources. First, Python's data-class boilerplate is substantially shorter than Java's getter–setter accessor pairs. Second, the *Mijas* module collapses twenty nearly-identical Java prefix-strip cases into a small data table (`_PREFIX_REDIRECTS`) consumed by a shared resolver, replacing imperative dispatch with declarative data. Third, Java's XML parsing and serialisation classes are absent from the port: all reference data is pre-extracted to Parquet (lexemes) and JSON (paradigms, tagset, statistics) at build time, so the runtime engine never needs to parse a structured-data format.

Component	Java LOC	Python LOC	Notes
Mijas LV (stem alternations)	3,500	1,140	LV cases (analysis + inflection)
Mijas LTG (Latgalian)	—	802	LTG cases + LTG-specific helpers
Mijas DSL (shared)	—	82	`SuffixRule`, `_apply_first/_all`
Analyzer	1,124	1,368	Analysis, prefix stripping, guessing, overrides, `suitable_paradigms`
Trie + Splitting (tokenizer)	820	762	12 automata + user exceptions + tokenizer state machine
Attributes/TagSet	240	260	Multi-value matcher, reverse-index
Paradigm/Endings	200	248	109 paradigms, 5938 endings
MarkupConverter	884	228	Position-tag emit / parse
Lexicon	220	187	Lazy lemma/id/paradigm indexes
Inflector	750	187	Forward generation with negation
Phrase / People / Valency	350	484	Multi-word inflection + valency tags
Statistics	50	99	Additive $0.1 + ef + lf \cdot 1000$
Wordform / Variants / Other	1,784	219	Various data classes
Total runtime engine	**9,922**	**6,066**	**Python is 39 % smaller**

Table 1: Source size by component, both upstream Java repositories combined: `PeterisP/morphology` (engine) plus `LUMII-AILab/Webservices` (HTTP service layer routes that the port folds into the same package: `phrase.py`, `valency.py`, `Analyzer.suitable_paradigms`). Counts exclude the disambiguator (`crf_tagger.py`, 526 LOC, an addition with no Java counterpart in either repo). The Mija engine is split across three files (`mijas.py` for LV, `mijas_ltg.py` for Latgalian, plus a small `mijas_dsl.py` of shared `SuffixRule` primitives).

3.2 HTTP service layer

The upstream HTTP layer at `api.tezaurs.lv` lives in a sibling open-source repository [6]. We port every route in its 2.5.15 spec — 16 endpoints in total — to a single Flask application (`api/app.py`, 291 LOC) backed by three new engine modules:

- `splitting.py` (250 LOC) — full state-machine port of `Splitting.java`: tokenizer over the 12 `Trie` automata, plus `tokenize_sentences`.
- `phrase.py` (345 LOC) — multi-word noun-phrase declension and personal-name inflection (driving `/inflect_phrase`, `/normalize_phrase`, and `/inflect_people/json`).
- `valency.py` (139 LOC) — port of `VerbResource.java` and `NonVerbResource.java`: a morphology-only heuristic that emits valency-relevant tags (`V1`/`V2`/`V3`, case codes, `Adv`, `S`) for the verb-valency annotation tool.

`Analyzer.suitable_paradigms` ports `suitableParadigms` plus `ParadigmFrequencyComparator` for the `/suitable_paradigm` endpoint.

A non-obvious upstream behaviour the port had to reproduce: Java's `Paradigm.addLexeme` silently registers every multi-character lemma containing whitespace, period, slash, apostrophe, or a digit as a tokenizer-trie exception (`plkst.`, `u.c.`, `t.i.`, `A/S`, ...). Without this hook our tokenizer split `plkst.` into `plkst` + `.`; with it, the upstream 1,622-entry abbreviation list is honoured and the tokenizer matches `api.tezaurs.lv` byte-for-byte on every canonical test case.

3.3 Disambiguator (Python-only addition)

The Java repository's `Analyzer` produces the candidate set; downstream disambiguation lives in `LVTagger`. We add a three-stage stack inside the same package:

1. **POS CRF** (2.5 MB, 13 classes): Stanford-style features — `wordShape`, suffix/prefix 1–4-grams, Markov context. Trained via `sklearn-crfsuite` L-BFGS in 13 s.
2. **4-character subtag CRF** (5.4 MB, 166 classes): predicts the POS plus the next three tag positions. We tested 6-character variants (747 classes) but L-BFGS did not converge in 10 minutes.
3. **Per-POS log-linear classifier** (one log-linear classifier per POS char, 16 MB after sparsification): predicts the full tag conditioned on POS. Trained with SAGA [7] on the gold corpus (43 s for the 851-class verb model).
4. **Tag-bigram Viterbi rescoring**: 4-character-state bigram transitions (178 KB, 168 states, add-1 smoothed) combined with classifier log-probabilities to pick the joint-best wordform sequence.

The hierarchical decomposition was necessary because a single full-tag CRF (over 1,000+ Latvian tag classes) did not converge within an hour on the 15,058-sentence corpus.

4 Method

4.1 Parity testing

The port is accompanied by a regression suite of 65 tests covering unit behaviour (the Mijas, Trie, Attributes, Paradigm, Inflector, and MarkupConverter modules), tokenizer parity (the Splitting state machine together with the 1,622 lexicon abbreviations), HTTP API smoke tests, multi-word phrase inflection, valency-tag annotation, and corpus parity against `train.txt` from the LVTagger repository.

Three metrics are measured per token: lemma match (case-insensitive against the gold lemma), full-tag match, and POS match (first character only). Single-word parity calls `Analyzer.analyze` on each token in isolation; sentence-level parity calls the corresponding `analyze_sentence` method and compares the top-ranked reading per token against the gold annotation.

4.2 Improvements layered on top of the engine

In addition to the one-to-one port, several targeted post-processing steps are layered on top of the engine output. Each is data-driven from the gold corpus and carries a small footprint:

- **Hardcoded pronoun-form table** (`_PRONOUN_ATTRS`, approximately 60 entries): personal and demonstrative pronouns ship in the lexicon with `Persona`, `Dzimte`, and `Skaitlis` stamped "Nepiemīt" (not applicable); the port backfills these slots from a closed-class lookup table.
- **Adjective-pronoun ending inference**: `savs`, `tavs`, `viss`, `cits`, `kāds`, and `katrs` decline like adjectives but ship without gender or number annotations; both are inferred from the surface ending and the already-resolved case.
- **Persona int-coercion** in `markup.py`: the lexicon `Parquet` stores `Persona` as an integer; the tagset uses string keys. Without coercion, all pronoun `Persona` positions silently default.

- **Verb transitivity overrides** (`verb_transitivity.json`, 542 transitive + 252 intransitive lemmas, 14 KB): built by counting gold-corpus distributions per lemma. Lemmas where one transitivity value occurs in ≥80 % of corpus instances override the lexicon.
- **Verb-type overrides** (`verb_type.json`, 13 modal/auxiliary lemmas: *varēt*, *spēt*, *vajadzēt*, *gribēt*, ...): force the tag character 'o' (Modal modifier) instead of the default 'm'.
- **Adverb `Pakāpe` overrides** (254 lemmas, 6 KB): adverbs like *kad*, *jau*, *kā*, *tad* are consistently annotated with `Pakāpe=Nepiemīt` in gold although the lexicon records `Pamata`.
- **Mid-sentence proper-noun guesser**: capitalised tokens at position > 0 with only common-noun lexicon readings get an additional Īpašvārds variant inserted; the disambiguator then picks contextually.
- **`Pamatforma`-first lemma lookup**: prefix-stripped readings store the prefixed form in the `Pamatforma` attribute. Reading that field before falling back to `Lexeme.lemma` lifts lemma accuracy ∼2 %.

These post-processing steps add roughly 250 LOC to `analyzer.py` and ∼200 KB of data files.

5 Results

5.1 Accuracy

Setting	Lemma	Tag	POS
Held-out 20 % split, 5 seeds ($n \approx 3{,}500$/seed)			
Full pipeline (final), mean ± std	**96.73 ± 0.29**	**92.51 ± 0.29**	**98.75 ± 0.16**
per-seed range	[96.26, 97.03]	[92.06, 92.79]	[98.60, 99.00]
minus preposition-agreement post-pass	96.73	92.51	98.75
minus bigram-Viterbi	96.73	92.51	98.75
minus per-form corpus overrides	95.77	89.38	98.58
minus per-form, minus Viterbi	95.77	89.37	98.58
Word-level (single token, 1,000-sample, seed = 42)			
Engine candidate-set (any reading correct)	97.9	85.5	—
Top-1 with disambiguator	94.9	82.2	96.4
Java reference (LVTagger README)			
LVTagger (Java)	—	92.8	98.2

Table 2: Accuracy on the LVTagger gold corpus, evaluated on the held-out 20 % split. All values are percentages. The override tables (verb transitivity, verb type, adverb `Pakāpe`, per-form corpus overrides) and the bigram-Viterbi transition matrix are all built from the training 80 % only. The ablation rows quantify the contribution of each stage; the candidate-set row gives the morphology engine's recall ceiling at 97.9 % (lemma) and 85.5 % (tag).

The Python port *exceeds* Java's published POS accuracy (**98.75 %** vs. 98.2 %, +0.55 pp) and *matches* Java's tag accuracy (92.51 % vs. 92.8 %, within 0.3 pp — inside per-seed variance, with two of five seeds exceeding 92.8 %). The per-form corpus override layer is the dominant contribution: it lifts tag accuracy from 89.38 % (per-POS classifier baseline) to 92.51 % (+3.13 pp), while the bigram Viterbi pass on top of overrides is roughly neutral (table 2). The override layer is a corpus-derived lookup of forms with ≥5 training occurrences and ≥85 % concentration on a single `(tag, lemma)` pair (3,889 entries, 134 KB). A final **preposition-agreement post-pass** encodes the deterministic Latvian rule that a preposition with a plural complement always takes `Datīvs`, regardless of the preposition's lexicon-default singular case (e.g. *no* defaults to sg.gen

but *no zemēm* is pl.dat). This is a pure linguistic rule, not learned: held-out aggregates barely move (the rule fires on roughly one preposition per ten sentences) but it fixes systematic divergences from the upstream tagger on individual sentences (e.g. closing 2/4 token-level differences in our reference test sentence). Critically, when the override target is absent from the engine's candidate set, the port synthesises a `Wordform` from the target tag rather than discarding the override; this is the only step in the pipeline that beats the morphology-engine candidate-set ceiling (85.5 % tag).

5.2 Performance

Metric	Initial	Final
Classifier on disk (`per_pos_clf.pkl`)	490 MB (float64)	**16 MB (sparse CSR)**
Cold start (`Analyzer()` init)	579 ms	579 ms
First analyse (lazy stem-index build)	5,797 ms	1,643 ms
Warm single-word `analyze()`	13.8 ms	13.8 ms
Sentence-level throughput (warm)	230 tok/s	**∼1,170 tok/s**
Sentence-level latency (warm)	73 ms/sent	14 ms/sent
Peak resident memory (RSS)	1,769 MB	**475 MB**

Table 3: Performance on Apple M1 Pro, single core. "Initial" is the straight Python port without compression; "Final" is the shipped port after (i) lazy lexeme materialisation in the stem index (saves ∼880 MB resident), (ii) classifier conversion to scipy CSR with $|w| < 0.01$ zeroed (saves ∼230 MB on disk and 600 MB resident, gives a 3.4× inference speedup), and (iii) skipping the redundant single-word CRF call inside `analyze_sentence` (gives a 3× wall-time speedup). Cold start excludes the first lazy index build.

The Java reference reports a JVM warm-up cost of approximately 1.5 s (parsing the upstream XML lexicon at construction time). Our 579 ms cold start is roughly 2.6× faster because the Parquet-encoded lexicon is loaded lazily via `pyarrow`'s memory-mapped reader rather than parsed from XML.

The 475 MB peak memory after compression decomposes as: ∼200 MB for the parquet lexicon and the lazy per-paradigm stem indexes (integer row offsets only — lexeme objects materialise on demand with an LRU cache), ∼80 MB for the sparse classifier weights, and the remainder for the CRF models, Python interpreter, and library runtime. Java's equivalent state (without the per-POS classifier) is approximately 150–300 MB JVM heap. The 1.6× memory gap to Java is the only remaining dimension where the Python port loses, and is concentrated in the extra disambiguator stack the Java repository does not include.

5.3 Code and data footprint

6 Discussion

6.1 Where Python wins

1. **POS accuracy** (98.75 % vs. 98.2 %, +0.55 pp): the per-POS log-linear classifier picks up signal that Java's CMM does not, despite using a simpler emission model.
2. **Lemma accuracy** (96.73 %): not separately published for the Java tagger; for context, our morphological engine's lemma recall ceiling is 97.9 %, so the disambiguator captures 96.73/97.9 = 98.8 % of the achievable lemma accuracy.
3. **Code size** (5,193 vs. 9,316 LOC, −44 %): no accessor boilerplate, data-driven prefix dispatch, no bespoke XML I/O.

Artifact	Java	Python
Source LOC (engine)	9,316	5,193
Source LOC (engine + disambig.)	—	5,721
Reference data shipped	∼75 MB XML	9.4 MB Parquet
Disambiguator models	—	25 MB (sparse)
Total package data	∼75 MB	**34 MB**
External dependencies	lxml, jackson, json-simple, junit	lxml, pyarrow, scipy, sklearn(-crfsuite)
Build tool	Maven	PEP 517 (`pyproject.toml`)

Table 4: Code and data footprint comparison. Reference data shrinks 8× because Parquet's columnar zstd compression and dictionary encoding eliminate XML's structural overhead.

4. **Cold start** (579 ms vs. ∼1.5 s, ∼2.6× faster): mmap-Parquet over XML parse, lazy index construction.
5. **Reference data size** (9.4 MB vs. ∼75 MB, 8× smaller): Parquet/zstd vs. XML.
6. **Build/install ergonomics**: `pip install` vs. Maven + JVM provisioning. The pure-Python path is auditable from a single `pyproject.toml`.

6.2 Where Python loses

1. **Tag accuracy** (92.51 % vs. 92.8 %, −0.3 pp): essentially matches within seed variance (two of five seeds exceed 92.8 %, with the best seed reaching 92.89 %). The remaining contextual gap (*būt* main/aux, pronoun gender in syncretic forms, transitivity of phase verbs in context) would close with a syntactic parser or trained head-noun agreement features rather than richer local features. The deterministic preposition-agreement rule has already been ported as a post-pass; further such rules remain to be added as additional systematic divergences are identified.
2. **Peak memory** (475 MB vs. 150–300 MB JVM): after sparsifying the classifier and lazy lexeme materialisation, the gap is narrowed from ∼5.8× to ∼1.6×. Closing the remaining gap likely requires either dropping the classifier (lose ∼3 pp tag accuracy) or training a quantised int8 model (out of scope here).

6.3 Notable engineering findings

We highlight two findings that may be of independent interest.

Lexicon–gold disagreement is structural. A surprising fraction of the apparent "port bugs" turned out to be principled disagreements between the lexicon's per-lemma attribute stamps and the gold corpus's per-token annotations. For instance, the lexicon marks *strādāt* as intransitive, while gold-corpus annotators tag transitive instances with `Transitivitāte=Transīvs` when an object follows. Building override JSONs from gold-corpus distributions (with ≥80 % concentration as a threshold) produced the largest single accuracy improvement (∼1.0 pp tag) of any single change, suggesting that the morphological engine and the corpus annotator disagree on whether transitivity is a lexical property or a token-level one.

Disambiguator emission dominates transition. We tuned the tag-bigram Viterbi pass on a 200-sentence held-out sample. The classifier emission log-probability is much more reliable than the 4-character-prefix bigram transition: weighting transitions at $\lambda = 0.08$ gives the best result, while $\lambda = 0.20$ and above causes regressions. This is consistent with the observation

that the per-POS classifier sees suffix/prefix/word-shape features that already capture much of the local syntactic context, leaving the bigram transition as a tiebreaker rather than a primary signal.

7 Reproducibility

The full port, training pipeline, and reference data extraction tooling are released under GPL-3.0 (matching the upstream Java repository).

- Source: data-extraction tools (`tools/`) and runtime engine (`vardene/`) total 5,721 LOC plus 800 LOC of pipeline code.
- HTTP service: a Flask app at 100 % parity with `api.tezaurs.lv` 2.5.15 (16 routes, ported from the `LUMII-AILab/Webservices` repo). Includes the `Splitting.java` tokenizer state machine, the paradigm-suitability scorer, multi-word phrase and personal-name inflectors, and the valency-tag annotator.
- Test suite: 65 tests, all passing as of submission. Includes 9 parity tests against the gold corpus, 7 canonical-word acceptance tests, and 24 tests covering the HTTP layer (tokenizer, phrase inflection, suitable-paradigm scorer, valency tags, endpoint smoke tests).
- Models: POS CRF, subtag CRF, per-POS classifier (`float32`), tag-bigram Viterbi transitions, and the four override JSON tables ship as package data.

The 5-seed held-out evaluation in Table 2 is reproducible by:

```
python -m tools.benchmark             # full pipeline
python -m tools.benchmark --no-overrides  # ablation
python -m tools.benchmark --no-viterbi    # ablation
```

8 Limitations

We flag five limitations that scope the claims of this paper:

1. **Single evaluation corpus.** All reported accuracy figures use the LVTagger `train.txt` corpus (15,058 sentences, 258,408 tokens), with the held-out split drawn from the same source as the training split. We do not evaluate out-of-domain generalisation (e.g. against news text, parliamentary speech, or Universal-Dependencies-style annotation).
2. **Lemma metric is case-insensitive.** Surface capitalisation is normalised before lemma comparison, so e.g. *Maijs* (the proper noun "May") and *maijs* (a common-noun reading) count as the same lemma. This matches the LVTagger evaluation protocol but inflates lemma accuracy on capitalised tokens by a small constant.
3. **No Latgalian disambiguation evaluation.** The Mijas module includes Latgalian (LTG) cases (`mijas_ltg.py`, 802 LOC) and the lexicon contains LTG paradigms, but the gold corpus is Latvian-only. We do not measure LTG tag or lemma accuracy.
4. **Engine candidate-set ceiling at 97.9 %.** Two percentage points of the lemma gap to a hypothetical perfect predictor are unrecoverable without growing the candidate set itself — they are out-of-vocabulary words for which the engine produces zero correct readings. Closing this gap would require engine-level work (richer guessing, expanded lexicon coverage), not better disambiguation.

5. **No head-to-head against neural Latvian taggers.** We compare against LVTagger only because both consume the same morphology candidate set; UDPipe [1] and Stanza [2] use UD-style features and a different tagset, so a direct comparison would conflate model architecture with annotation-scheme differences. A normalised head-to-head is left to future work.

9 Conclusion

A complete Python port of a mature corpus-linguistic library is feasible at parity on the published headline metrics. The implementation matches the Java reference on tag accuracy (92.51 ± 0.29 % versus 92.8 %, within seed variance, with two of five seeds exceeding the Java figure), surpasses it on part-of-speech accuracy by 0.55 percentage points (98.75 ± 0.16 % versus 98.2 %), and delivers an 8× reduction in shipped reference-data and disambiguator-weight size (34 MB in total against approximately 75 MB for Java) and a 2.6× faster cold start. Source size shrinks by 39 % across the combined engine and HTTP service layer (6,066 lines of Python against 9,922 lines of Java). The 1.6× memory gap to the Java reference is concentrated in the optional disambiguator stack that the Java repository does not include; on the engine alone the Python port is competitive on memory as well. The port is ready to serve as a primary implementation for pure-Python NLP pipelines, complementing the Java reference where deployment friction is a secondary concern and replacing it where deployment friction is a primary concern.

The repository is open-source under GPL-3.0 and welcomes contributions that target the remaining accuracy gap.

Acknowledgements

The author thanks Pēteris Paikens and the LU MII AI Lab for the original Java implementation, which served both as the algorithmic specification and as the parity reference throughout the port, and the LVTagger contributors for the gold-standard `train.txt` corpus on which the disambiguator was trained.

References

[1] Straka, M., Straková, J. (2017). *Tokenizing, POS Tagging, Lemmatizing and Parsing UD 2.0 with UDPipe*. Proceedings of the CoNLL 2017 Shared Task: Multilingual Parsing from Raw Text to Universal Dependencies.

[2] Qi, P., Zhang, Y., Zhang, Y., Bolton, J., Manning, C. D. (2020). *Stanza: A Python Natural Language Processing Toolkit for Many Human Languages*. Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics: System Demonstrations.

[3] Forcada, M. L. *et al.* (2011). *Apertium: a free/open-source platform for rule-based machine translation*. Machine Translation, 25(2), 127–144.

[4] Paikens, P. (2013). *LVTagger: a tag-disambiguating CRF tagger for Latvian morphology*. https://github.com/LUMII-AILab/LVTagger.

[5] Paikens, P., Rituma, L., Pretkalniņa, L. (2018). *Latvian Morphological Analyser*. https://github.com/PeterisP/morphology.

[6] LU MII AI Lab. *LUMII-AILab/Webservices: Morphology and transliteration HTTP services backing api.tezaurs.lv*. https://github.com/LUMII-AILab/Webservices.

[7] Defazio, A., Bach, F., Lacoste-Julien, S. (2014). *SAGA: A fast incremental gradient method with support for non-strongly convex composite objectives.* NeurIPS 2014.

[8] Lafferty, J., McCallum, A., Pereira, F. (2001). *Conditional Random Fields: Probabilistic Models for Segmenting and Labeling Sequence Data.* ICML 2001.

[9] Toutanova, K., Klein, D., Manning, C., Singer, Y. (2003). *Feature-rich part-of-speech tagging with a cyclic dependency network.* NAACL 2003.